



Rhythm-Based Timing Game: Milestone 5

[SWE450](#)

08/26/2025

[Elijah Brandner](#)

Grand Canyon University

College of Engineering and Technology

Software Engineering Program

Embedded Systems 2

Table of Contents

<i>Table of Contents</i>	<i>1</i>
<i>Project Demonstration Link</i>	<i>2</i>
<i>GitHub Repo Link</i>	<i>3</i>
<i>System Design</i>	<i>4</i>
<i>Application Logic Design</i>	<i>6</i>
<i>Risk Matrix.....</i>	<i>8</i>
<i>Test Plan</i>	<i>9</i>
<i>Known Issues.....</i>	<i>10</i>
<i>Misc Documentation</i>	<i>11</i>

Project Demonstration Link

<https://youtu.be/a1bEhrv9Sis>

GitHub Repo Link

<https://github.com/elijahbrandner/de10-rhythm-game.git>

System Design

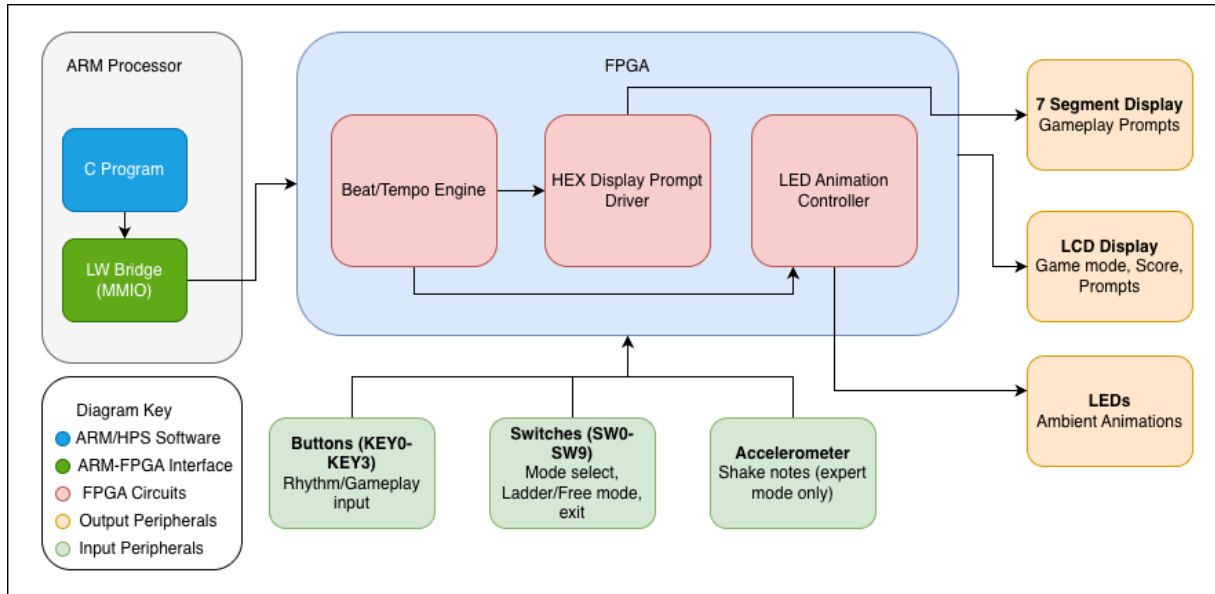


Figure 1: System Design Diagram for Rhythm-Based Timing Game

The initial system is composed of an embedded C application running on the ARM HPS and a small set of custom FPGA circuits that provide beat-synchronous timing and hardware-accelerated visual output. The ARM HPS serves as the primary control unit, handling game state management, sequence generation, scoring calculations, user input coordination, and LCD output.

User inputs are provided through push buttons (KEY0-KEY3), board switches, and the onboard accelerometer. Outputs are provided through the seven-segment displays (HEX0-HEX3), LEDs, and the LCD display.

The FPGA fabric hosts a minimal set of custom hardware circuits, including a beat and tempo engine and hardware display controllers for the seven-segment displays and LEDs. The first circuit is a beat and tempo engine that generates precise timing signals based on the selected game mode's BPM. This engine ensures deterministic rhythm timing independent of software delays. The second circuit is a seven-segment display driver responsible for rendering gameplay prompts, including individual lane indicators and full-display flash patterns for shake prompts. An LED animation controller is implemented in FPGA to ensure smooth, beat-synchronized visual effects independent of software timing.

Communication between the ARM HPS and the FPGA occurs through memory-mapped registers over the lightweight HPS-FPGA bridge, allowing the software to configure tempo, display patterns, and configure tempo, display patterns, and synchronize visual output through hardware-controlled timing, while maintaining hardware level accuracy

All gameplay logic, input handling, scoring, and state transitions are implemented entirely in C on the ARM HPS, keeping the FPGA role intentionally limited to timing accuracy and visual synchronization. The FPGA does not provide timing feedback to the HPS. Instead, the HPS maintains its own software-based timing for gameplay and scoring, while the FPGA independently generates beat-synchronized visual cues.

Application Logic Design

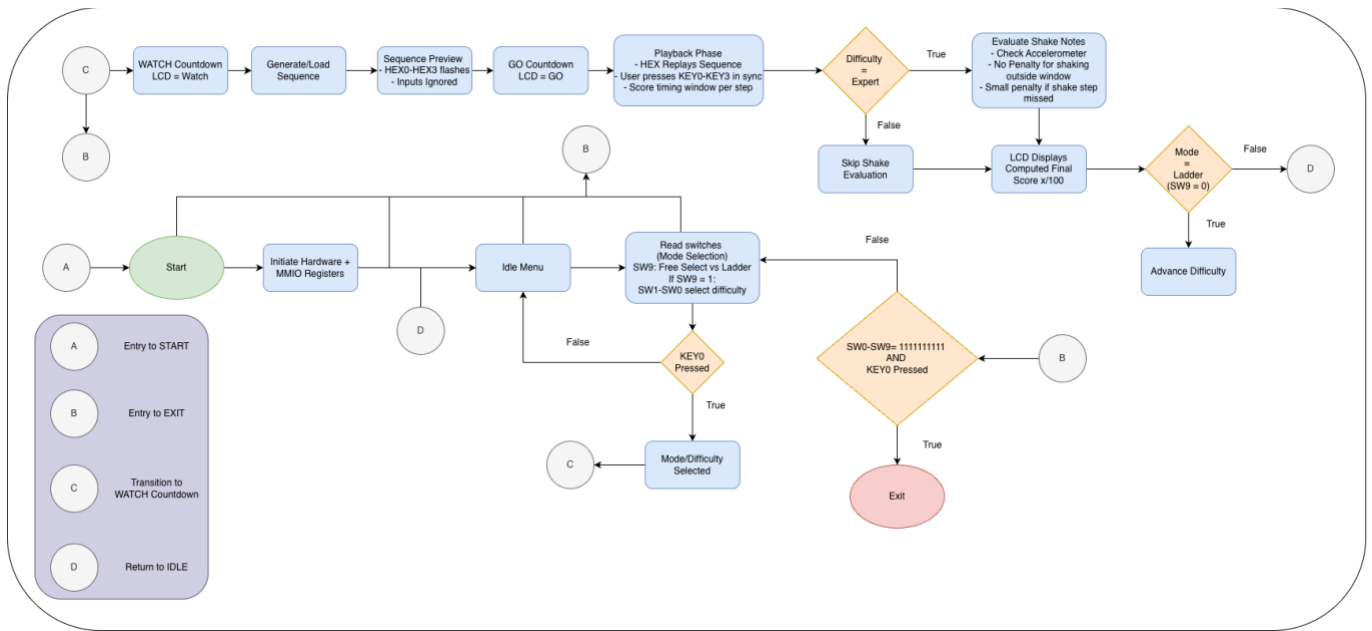


Figure 2: Application Logic Flowchart for Rhythm-Based Timing Game

The application begins in an idle state where animated LED patterns are displayed and the LCD presents instructions. The user selects ladder or free-select mode using switches and initiates gameplay by pressing KEY0.

A countdown phase follows, during which the seven segment displays a fixed-duration countdown phase while the LCD displays a “Watch” prompt. The system then enters a sequence preview phase where a fixed-length prompt sequence is displayed once on the seven-segment displays. During this phase, all user input is ignored.

After a second countdown displaying “Go,” the application enters the playback phase. The same sequence is replayed at the same tempo, and user inputs are now evaluated. Each step begins when the game logic activates the corresponding lane on the seven-segment display. The timing offset between the displayed prompt and the user’s response is evaluated in software to

determine the score awarded for that step. Incorrect button presses apply a penalty once per step, while missing inputs result in no points.

In Expert mode, specific steps are designated as shake prompts, during which the accelerometer is monitored for shake events. Shake inputs are scored using a larger timing window and are ignored outside of designated shake steps.

Once the playback phase completes, the application transitions to a result state where the final score and performance rating are displayed on the LCD. The scoring window begins at the start of each step and extends for a defined portion of the beat duration. User inputs are evaluated based on their timing offset from the start of the step.

In ladder mode, the user may proceed to the next difficulty, while free-select mode returns to the selection screen. A defined switch and button combination allows the user to exit the application.

The accelerometer is interfaced through the HPS using I2C communication with the ADXL345 sensor. Shake detection is performed in software by comparing changes in acceleration values across successive samples. A configurable threshold is used to detect shake notes, and these events are only considered valid during designated shake steps in Expert mode.

Risk Matrix

ID	Category	Description	Severity	Mitigation
1	Technical	Timing jitter if rhythm is software-driven	H	Reduction – Use FPGA – based beat/tempo engine
2	Technical	Button bounce causing false inputs	M	Reduction – Software edge detection and per-step input gating
3	Resource	Accelerometer noise triggering false shakes	M	Reduction – Apply threshold + only sample during shake steps + require N consecutive samples
4	Integration	HPS-FPGA register misconfiguration	H	Avoidance – Start with minimal register map and test early
5	Usability	User confused by visual prompts	M	Acceptance – Use simple HEX patterns and clear WATCH/GO phases
6	Scope	Feature creep (Extra modes, animations)	L	Avoidance – Lock core gameplay early, treat extras as optional

Test Plan

#	Title	Summary	Steps	Data	Expected Result	Status
1	Mode selection	Verify switches select mode	1. Toggle SW9, SW1-SW0	Switch States	LCD Updates mode	Completed
2	Countdown Timing	Verify BPM-based HEX countdown	Start game	BPM Modes	Correct countdown speed	Completed
3	Sequence Preview	Preview plays once	Start session	Generated Sequence	Hex displays sequence	Completed
4	Playback Scoring	Inputs scored correctly	Press keys	Key presses	Score is normalized to a 0-100 scale and rating text is displayed.	Completed
5	Shake detection	Shake only counted in expert	Shake board	Accelerometer	Shake only counted during designated shake steps in Expert mode	Completed

Known Issues

NOTE: Add rows to the table below as needed:

ID	Description	Plan of Address
1	Accelerometer thresholds	Calibrate empirically during testing
2	HEX patterns hard to distinguish	Adjust segment choice or brightness
3	Timing windows too strict	Tune thresholds after initial playtesting
4	LED animation conflicts without prompts	Separate animations and prompt control paths
5	Exit condition hard to discover	Document exit combo clearly in LCD text
6	LED pulse slightly misaligned with HEX prompt timing	Accepted as design tradeoff to preserve gameplay feel

Misc Documentation

Project Overview:

This project proposes the design and implementation of an embedded rhythm and timing memory game developed on the Terasic DE10-Standard board. The application combines precise timing, user interaction, and hardware accelerated visual feedback to challenge users to observe, memorize, and accurately reproduce rhythmic input sequences. The system leverages the ARM Hard Processor System (HPS) for application control and scoring logic, while offloading timing-critical and display driven operations to custom FPGA circuits.

The system intentionally uses a dual-timing approach, where the FPGA generates visually synchronized beat signals while the HPS independently manages gameplay timing and scoring windows. The separation simplifies hardware design while maintaining responsive gameplay.

The game operates, by presenting a timed sequence of visual prompts using the seven-segment displays, followed by a playback phase in which the user must reproduce the sequence using physical push buttons and, in advanced modes, accelerometer-based shake gestures. Game difficulty is determined by tempo and input complexity, with multiple modes including Easy, Medium, Hard and Expert. Visual feedback is enhanced through animated LED patterns, while game state, score, and performance feedback are displayed on the onboard LCD.

This project demonstrates core embedded systems concepts including hardware/software partitioning, real time input handling, finite state machines, memory-mapped I/O, and FPGA accelerated by timing logic. The design is developed incrementally across milestones, beginning with a fully specified system architecture and application flow in this proposal.

Functional Requirements:

- As a user, I want to select between ladder modes and free -select mode using switches so that I can either progress through difficulties or play a specific mode directly.
- As a user, I want to start a game session by pressing a button so that gameplay begins only when I am ready.
- As a user, I want to observe a visual sequence preview before gameplay so that I can memorize the order and rhythm of inputs.
- As a user, I want the game to replay the same sequence during gameplay so that I can attempt to match the sequence using buttons in rhythm.
- As a user, I want each input to be scored based on how accurately and quickly I respond so that timing precision is rewarded.
- As a user, I want incorrect button presses to negatively affect my score so that accuracy matters, not just speed.
- As a user, I want advanced difficulty modes to include shake-based inputs using the accelerometer so that gameplay becomes more physically interactive.
- As a user, I want accidental shakes outside of shake prompts to be ignored so that I am not unfairly penalized.
- As a user, I want my final score to be normalized out of 100 and displayed with performance feedback so that results are easy to understand.
- As a user, I want animated LEDs to run throughout the application so that the system provides continuous visual feedback even outside of gameplay.
- As a user, I want a defined exit condition using switches and a button so that I can safely terminate the application.

FPGA Circuit Design

To minimize hardware complexity while still meeting real-time timing and visual synchronization requirements, the FPGA portion of the system is intentionally limited to three dedicated hardware circuits. These circuits are designed to operate independently of game logic, which remains fully implemented in software on the ARM HPS. The FPGA circuits serve as timing-accurate and visually synchronized support modules for the application.

Beat and Tempo Engine (beat_engine.v)

```
1  module beat_engine (  
2      input wire      clk,  
3      input wire [3:0] tempo_sel,  
4      input wire      enable,  
5      input wire      rst,  
6      output wire     beat_tick  
7  );  
8  
9      function [31:0] bpm_cycles;  
10         input [3:0] sel;  
11         begin  
12             case (sel)  
13                 4'd0: bpm_cycles = 32'd66666666; // 45 BPM  
14                 4'd1: bpm_cycles = 32'd50000000; // 60 BPM  
15                 4'd2: bpm_cycles = 32'd40000000; // 75 BPM  
16                 default: bpm_cycles = 32'd50000000; // default 60 BPM  
17             endcase  
18         end  
19     endfunction  
20  
21     reg [31:0] counter = 0;  
22     reg      tick_r = 0;  
23  
24     wire [31:0] period = bpm_cycles(tempo_sel);  
25  
26     always @(posedge clk) begin  
27         tick_r <= 0;  
28  
29         if (rst || !enable) begin  
30             counter <= 0;  
31         end else begin  
32             if (counter >= period - 1) begin  
33                 counter <= 0;  
34                 tick_r <= 1;  
35             end else begin  
36                 counter <= counter + 1;  
37             end  
38         end  
39     end  
40  
41     assign beat_tick = tick_r;  
42  
43 endmodule
```

The Beat and Tempo Engine is responsible for generating a stable, deterministic beat signal based on the selected game tempo. This circuit

operates independently of the operating system and software scheduling delays, ensuring consistent timing regardless of HPS workload.

The engine accepts configuration parameters from the ARM HPS via memory-mapped registers, including tempo selection (45, 60, 75 BPM) and enable/reset control signals. Internally, the circuit derives a beat tick from the 50 MHz FPGA system clock and produces a single-cycle beat_tick pulse when the counter reaches the calculated interval. This beat signal serves as the timing reference for LED animations and other beat-synchronized visual feedback within the FPGA.

By offloading beat generation to hardware, the system eliminates timing jitter that could otherwise impact rhythm-based gameplay. The circuit does not evaluate user input or game rules. It functions strictly as a hardware timing reference. The beat_tick signal is used internally by FPGA modules and is not directly read by the HPS.

Seven-Segment Display (hex_prompt.v)

```
1 module hex_prompt (  
2     input wire [1:0] lane_select,  
3     input wire [1:0] mode,  
4     input wire enable,  
5     output wire [31:0] hex3_hex0,  
6     output wire [15:0] hex5_hex4  
7 );  
8  
9 // internal active-high segment pattern  
10 // top-level still inverts for DE10 active-low HEX outputs  
11 localparam [6:0] SEG_EIGHT = 7'b1111111;  
12  
13 reg [31:0] hex_r;  
14  
15 always @(*) begin  
16     hex_r = 32'd0;  
17  
18     if (enable) begin  
19         // mode 2 = preview/playback: only active lane shown  
20         if (mode == 2'd2) begin  
21             case (lane_select)  
22                 2'd0: hex_r[ 6: 0] = SEG_EIGHT;  
23                 2'd1: hex_r[14: 8] = SEG_EIGHT;  
24                 2'd2: hex_r[22:16] = SEG_EIGHT;  
25                 2'd3: hex_r[30:24] = SEG_EIGHT;  
26             endcase  
27         end else begin  
28             // idle/watch/go/results: light all four  
29             hex_r[ 6: 0] = SEG_EIGHT;  
30             hex_r[14: 8] = SEG_EIGHT;  
31             hex_r[22:16] = SEG_EIGHT;  
32             hex_r[30:24] = SEG_EIGHT;  
33         end  
34     end  
35 end  
36  
37 assign hex3_hex0 = hex_r;  
38 assign hex5_hex4 = 16'd0;  
39  
40 endmodule
```

The Seven-Segment Display Prompt Driver controls the HEX0-HEX3 displays used to present the gameplay sequence during both the preview and the playback phases. Rather than rendering multiple unique symbols, the circuit displays a single, consistent segment pattern that is selectively enabled on one of the four seven-segment displays at a time. Each active display corresponds to a specific input lane (KEY0-KEY3), allowing the user to visually identify which button should be pressed.

The ARM HPS determines which display should be active for each step in the sequence and writes a simple control value to the FPGA indicating the target display. The FPGA then applies the fixed segment pattern to the selected

HEX output, ensuring clean and stable visual presentation synchronized to the tempo of the game.

By using a uniform display pattern across all lanes, we reduce hardware decoding complexity while maintaining clear and intuitive input cues. This design aligns with the project goal of keeping FPGA logic minimal and purpose-driven.

LED Animation Controller (led_controller.v)

```
1 module led_controller (
2     input wire clk,
3     input wire beat_tick,
4     input wire [1:0] mode,
5     input wire enable,
6     input wire rst,
7     output wire [9:0] leds
8 );
9
10 localparam integer IDLE_STEP_TICKS = 5000000; // 1 second at 50 MHz
11 localparam integer PULSE_PHASE_TICKS = 6000000; // ~120 ms per phase
12
13 // -----
14 // Idle bounce state (4-LED bar)
15 // -----
16 reg [25:0] idle_counter = 0;
17 reg [2:0] idle_pos = 0; // 0..6 valid for 4-wide bar across 10 LEDs
18 reg idle_dir = 0; // 0 = move right, 1 = move left
19
20 // -----
21 // Gameplay pulse state
22 // -----
23 reg [22:0] pulse_counter = 0;
24 reg [2:0] pulse_phase = 0;
25 reg pulse_active = 0;
26
27 // -----
28 // Blink state
29 // -----
30 reg blink = 0;
31
32 reg [9:0] leds;
33
34 always @(posedge clk) begin
35     if (rst || !enable) begin
36         idle_counter <= 0;
37         idle_pos <= 0;
38         idle_dir <= 0;
39
40         pulse_counter <= 0;
41         pulse_phase <= 0;
42         pulse_active <= 0;
43
44         blink <= 0;
45     end else begin
46         case (mode)
47             //
```

```

46 case (mode)
47     // -----
48     // mode 0: off
49     // -----
50     2'd0: begin
51         pulse_counter <= 0;
52         pulse_phase <= 0;
53         pulse_active <= 0;
54     end
55
56     // -----
57     // mode 1: 4-LED bar bounce
58     // -----
59     2'd1: begin
60         if (idle_counter >= IDLE_STEP_TICKS - 1) begin
61             idle_counter <= 0;
62
63             if (!idle_dir) begin
64                 if (idle_pos == 3'd0) begin
65                     idle_dir <= 1'b1;
66                     idle_pos <= 3'd5;
67                 end else begin
68                     idle_pos <= idle_pos + 1'b1;
69                 end
70             end else begin
71                 if (idle_pos == 3'd0) begin
72                     idle_dir <= 1'b0;
73                     idle_pos <= 3'd1;
74                 end else begin
75                     idle_pos <= idle_pos - 1'b1;
76                 end
77             end
78         end else begin
79             idle_counter <= idle_counter + 1'b1;
80         end
81     end
82
83     // -----
84     // mode 2: gameplay pulse
85     // -----
86     2'd2: begin
87         if (beat_tick) begin
88             pulse_active <= 1'b1;
89             pulse_phase <= 3'd0;
90             pulse_counter <= 0;
91         end else if (pulse_active) begin
92             if (pulse_counter >= PULSE_PHASE_TICKS - 1) begin
93                 pulse_active <= 1'b1;
94                 pulse_phase <= 3'd0;
95                 pulse_counter <= 0;
96             end else if (pulse_phase == 3'd3) begin
97                 pulse_active <= 1'b0;
98             end else begin
99                 pulse_phase <= pulse_phase + 1'b1;
100             end
101         end else begin
102             pulse_counter <= pulse_counter + 1'b1;
103         end
104     end
105
106     // -----
107     // mode 3: full blink on beat
108     // -----
109     2'd3: begin
110         if (beat_tick)
111             blink <= ~blink;
112     end
113
114 endcase
115
116 end
117
118 always @(*) begin
119     leds = 10'd0;
120
121     if (!enable) begin
122         leds = 10'd0;
123     end else begin
124         case (mode)
125             2'd0: leds = 10'd0;
126
127             // 4-wide bar starting at idle_pos
128             2'd1: leds = (10'b0000001111 << idle_pos);
129
130             // pulse / ripple
131             2'd2: begin
132                 if (!pulse_active) begin
133                     leds = 10'd0;
134                 end else begin
135                     case (pulse_phase)
136

```

```

109 //
110 2'd3: begin
111     if (beat_tick)
112         blink <= ~blink;
113     end
114 endcase
115 end
116
117
118
119 always @(*) begin
120     leds = 10'd0;
121
122     if (!enable) begin
123         leds = 10'd0;
124     end else begin
125         case (mode)
126             2'd0: leds = 10'd0;
127
128             // 4-wide bar starting at idle_pos
129             2'd1: leds = (10'b0000001111 << idle_pos);
130
131             // pulse / ripple
132             2'd2: begin
133                 if (!pulse_active) begin
134                     leds = 10'd0;
135                 end else begin
136                     case (pulse_phase)
137                         3'd0: leds = 10'b0000110000;
138                         3'd1: leds = 10'b0001110000;
139                         3'd2: leds = 10'b0011001100;
140                         3'd3: leds = 10'b0110000110;
141                         default: leds = 10'd0;
142                     endcase
143                 end
144             end
145
146             2'd3: leds = blink ? 10'b1111111111 : 10'd0;
147
148             default: leds = 10'd0;
149         endcase
150     end
151 end
152
153 assign ledr = leds;
154
155 endmodule

```

The LED animation controller manages animated LED patterns used throughout the application, including idle animations, beat synchronized pulse effects during gameplay, and blinking patterns used for feedback. The circuit uses the `beat_tick` signal generated by the Beat and Tempo Engine to ensure that LED animations remain precisely aligned with the selected tempo.

The ARM HPS selects the active LED animation mode via memory-mapped control registers based on the current application state (idle, preview, playback, results). Once a mode is selected, the FPGA autonomously drives the LED outputs using internal counters and beat synchronization, allowing smooth animations without requiring continuous software intervention.

This design ensures consistent visual feedback while reducing software complexity and processor overhead.

FPGA Integration Module (`rhythm_fpga_circuits.v`)

```

1  module rhythm_fpga_circuits (
2      input wire      clk,
3      input wire [31:0] ctrl,
4      output wire [31:0] hex3_hex0,
5      output wire [15:0] hex5_hex4,
6      output wire [9:0]  ledr
7  );
8
9      wire [1:0] lane    = ctrl[1:0];
10     wire [1:0] mode    = ctrl[3:2];
11     wire [3:0] tempo    = ctrl[7:4];
12     wire      enable    = ctrl[8];
13     wire      rst       = ctrl[9];
14
15     wire beat_tick;
16
17     beat_engine u_beat (
18         .clk(clk),
19         .tempo_sel(tempo),
20         .enable(enable),
21         .rst(rst),
22         .beat_tick(beat_tick)
23     );
24
25     hex_prompt u_hex (
26         .lane_select(lane),
27         .mode(mode),
28         .enable(enable),
29         .hex3_hex0(hex3_hex0),
30         .hex5_hex4(hex5_hex4)
31     );
32
33     led_controller u_led (
34         .clk(clk),
35         .beat_tick(beat_tick),
36         .mode(mode),
37         .enable(enable),
38         .rst(rst),
39         .ledr(ledr)
40     );
41
42     endmodule

```

The `rhythm_fpga_circuits` module serves as the integration layer that connects the three independent FPGA hardware circuits: Beat and Tempo engine, Seven-Segment prompt driver, and the LED Animation controller. Rather than combining all logic into a single monolithic module, the design intentionally separates functionality into dedicated submodules and uses this wrapper to coordinate signal routing.

This module receives a 32-bit control word from the ARM HPS and decodes it into configuration fields:

Bits	Function
[1:0]	HEX Lane selection
[3:2]	LED animation mode

[7:4]	Tempo selection
[8]	Enable
[9]	Reset

These fields are routed to the appropriate submodules. The beat engine generates a `beat_tick` signal that is internally distributed to the LED controller to synchronize beat-based animations.

The module contains no gameplay logic, being responsible only for signal routing and hardware composition. This design preserves modularity, maintains a clear hardware-software boundary, and simplifies future expansion.

Hardware-Software Integration Summary

Together, these three FPGA circuits provide precise timing and synchronized visual output while intentionally avoiding implementation of gameplay logic in hardware. All game state management, scoring, input handling, and mode progression are handled in C on the ARM HPS. Communication between software and hardware occurs through a minimal 32-bit control word transmitted over the lightweight HPS-FPGA bridge. This interface provides lane selection, tempo selection, animation mode control, and enable/reset signaling.

This architecture achieves a balanced hardware-software partitioning strategy:

- Hardware provides deterministic timing and real-time visual control
- Software maintains flexibility, game logic, and state management

The result is a simplified FPGA design that preserves deterministic timing behavior while minimizing hardware complexity.